Spice up your SQL

Intro to Window Functions



Contact

Koen Verbeeck

@Ko_Ver

@koenv.bsky.social

SQLkover.com





Thank you, partners









































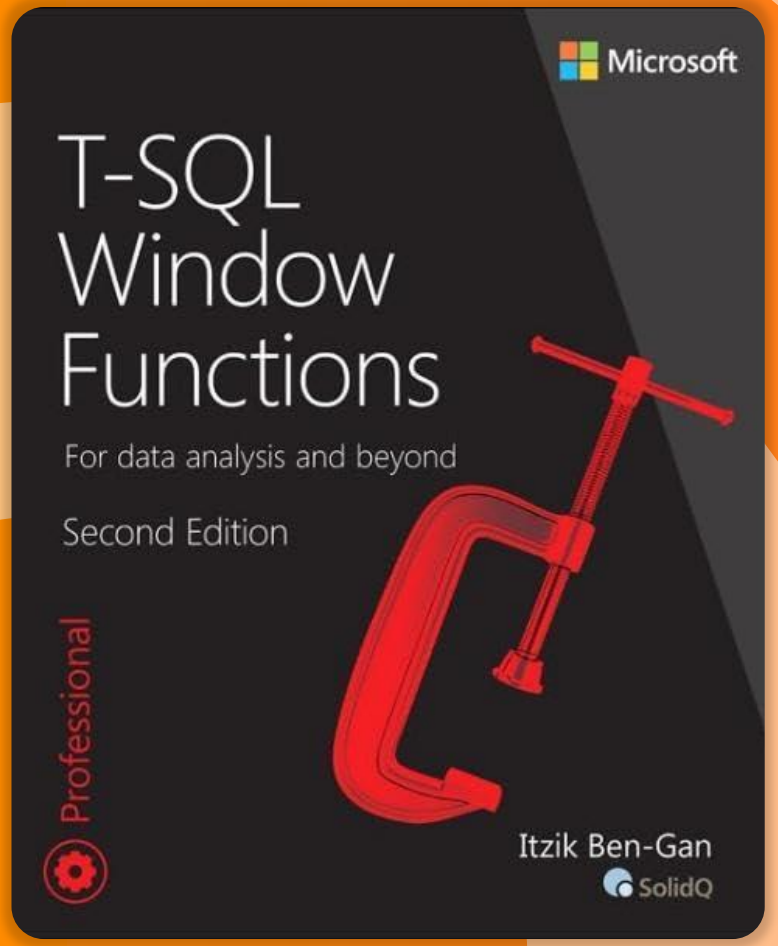








Outline



01 Why window functions?

02 What are window functions?

03 Common patterns

04 Performance

05 Conclusion

Why window functions?



Running totals: on your marks...

Same result, three very different engines



`SUM(...) OVER (ORDER BY ...)`

Window function — one pass over the data

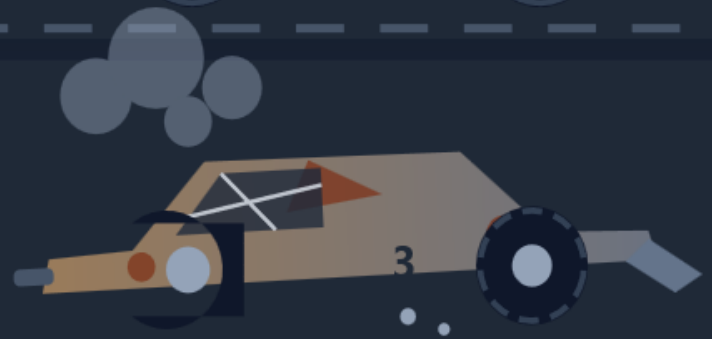
Readable, set-based, and it actually scales



`JOIN ... ON p.d <= m.d`

Triangular self-join — gets there eventually

Work grows with the square of the row count



`DECLARE CURSOR ... FETCH NEXT`

Row-by-agonizing-row — still in the pit lane

Easy to read, painful to run

What are window functions?



Window functions

```
window function
OVER (
    [PARTITION BY expression]
    [ORDER BY expression]
    [ROWS | RANGE BETWEEN frame bound AND frame bound]
     )
```

EXAMPLE 1

Available since SQL Server 2005

```
SELECT DISTINCT
      SalesYear          = YEAR(OrderDate)
     ,SalesPerYear       = SUM(SalesAmount) OVER (PARTITION BY
                                                  YEAR(OrderDate))
     ,SalesGrandTotal    = SUM(SalesAmount) OVER ()
FROM AdventureWorksDW2019.dbo.FactInternetSales
ORDER BY SalesYear;
```

	SalesYear	SalesPerYear	SalesGrandTotal
1	2010	43421,0364	29358677,2207
2	2011	7075525,9291	29358677,2207
3	2012	5842485,1952	29358677,2207
4	2013	16351550,34	29358677,2207
5	2014	45694,72	29358677,2207

Logical query processing order

1. FROM
   - ON (used in joins)
   - OUTER/INNER/FULL/CROSS used in joins and/or APPLY
   - PIVOT/UNPIVOT
2. WHERE
3. GROUP BY
   - [CUBE / ROLLUP]
4. HAVING
5. SELECT
   - Calculating expressions
   - DISTINCT
6. ORDER BY
7. TOP / OFFSET-FETCH

EXAMPLE 2

```sql
SELECT
     tmp.CustomerAlternateKey
    ,tmp.SalesAmount
    ,CustomerRank = RANK() OVER (ORDER BY tmp.SalesAmount DESC)
FROM
( -- first calculate the sales amount for each customer
    SELECT
         c.CustomerAlternateKey
        ,SalesAmount = SUM(f.SalesAmount)
    FROM dbo.FactInternetSales f
    JOIN dbo.DimCustomer       c ON c.CustomerKey = f.CustomerKey
    GROUP BY c.CustomerAlternateKey
) tmp;
```

```
SELECT
     tmp.CustomerAlternateKey
    ,tmp.SalesAmount
    ,CustomerRank = RANK() OVER (ORDE
FROM
( -- first calculate the sales amount
    SELECT
         c.CustomerAlternateKey
        ,SalesAmount = SUM(f.SalesAmo
    FROM dbo.FactInternetSales f
    JOIN dbo.DimCustomer        c ON c
    GROUP BY c.CustomerAlternateKey
) tmp;
```

Results | Messages

	CustomerAlternateKey	SalesAmount	CustomerRank
1	AW00012301	13295,38	1
2	AW00012132	13294,27	2
3	AW00012308	13269,27	3
4	AW00012131	13265,99	4
5	AW00012300	13242,70	5
6	AW00012321	13215,65	6
7	AW00012124	13195,64	7
8	AW00012307	13173,19	8
9	AW00012296	13164,64	9
10	AW00011433	12909,6682	10
11	AW00011439	12489,1696	11
12	AW00011241	11469,1882	12
13	AW00011417	11248,4582	13
14	AW00011420	11200,7696	14

EXAMPLE 3

Since SQL Server 2012 - real window functions

```
WITH CTE_source AS (
    SELECT
         YearMonth      = YEAR(OrderDate) * 100 + MONTH(OrderDate)
        ,[Year]         = YEAR(OrderDate)
        ,OrderQuantity  = SUM(OrderQuantity)
    FROM dbo.FactResellerSales
    GROUP BY YEAR(OrderDate) * 100 + MONTH(OrderDate)
            ,YEAR(OrderDate)
)
SELECT
     YearMonth
    ,OrderQuantity
    ,RunningTotal = SUM(OrderQuantity) OVER (PARTITION BY [Year]
                                             ORDER BY YearMonth
                                             ROWS BETWEEN UNBOUNDED PRECEDING
                                                      AND CURRENT ROW
                                            )
FROM CTE_source
ORDER BY YearMonth;
```

Since SQL Server 2012 - real window functions

```
WITH CTE_source AS (
    SELECT
        YearMonth       = YEAR(OrderDate) * 100 +
       ,[Year]          = YEAR(OrderDate)
       ,OrderQuantity   = SUM(OrderQuantity)
    FROM dbo.FactResellerSales
    GROUP BY YEAR(OrderDate) * 100 + MONTH(OrderD
            ,YEAR(OrderDate)
)
SELECT
     YearMonth
    ,OrderQuantity
    ,RunningTotal = SUM(OrderQuantity) OVER (PART
                                             ORDE
                                             ROWS

                                                 )
FROM CTE_source
ORDER BY YearMonth;
```

Results | Messages

	YearMonth	OrderQuantity	RunningTotal
1	201012	820	820
2	201101	2053	2053
3	201103	2754	4807
4	201105	5208	10015
5	201107	852	10867
6	201108	3774	14641
7	201109	1260	15901
8	201110	2965	18866
9	201111	2204	21070
10	201112	7502	28572
11	201201	11044	11044
12	201202	8868	19912
13	201203	5355	25267
14	201204	8075	33342
15	201205	6342	39684
16	201206	3288	42972

How does it work?

Results | Messages

	YearMonth	OrderQuantity	RunningTotal
1	201012	820	820
2	201101	2053	2053
3	201103	2754	4807
4	201105	5208	10015
5	201107	852	10867
6	201108	3774	14641
7	201109	1260	15901
8	201110	2965	18866
9	201111	2204	21070
10	201112	7502	28572
11	201201	11044	11044
12	201202	8868	19912
13	201203	5355	25267
14	201204	8075	33342
15	201205	6342	39684
16	201206	3288	42972

The **PARTITION BY** defines the windows, one for each year

The **ORDER BY** sorts the rows within the window. Here they're sorted by month

The **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** frame extent sum all previous rows and the current row
11044 + 8868 + 5355
= 25267

Wut?

```
WITH CTE_source AS (
…
)
SELECT
      YearMonth
     ,OrderQuantity
     ,LastOrderValue = LAST_VALUE(OrderQuantity)
                       OVER (PARTITION BY [Year]
                       ORDER BY YearMonth)
FROM CTE_source
ORDER BY YearMonth;
```

The default frame extent is
`UNBOUNDED PRECEDING AND CURRENT ROW`

	YearMonth	OrderQuantity	LastOrderValue
1	201012	820	820
2	201101	2053	2053
3	201103	2754	2754
4	201105	5208	5208
5	201107	852	852
6	201108	3774	3774
7	201109	1260	1260
8	201110	2965	2965
9	201111	2204	2204
10	201112	7502	7502

Wut?

```
SELECT
      [Year]          = YEAR(OrderDate)
     ,[Sales Amount] = SUM(SalesAmount)
     ,GrandTotal     = SUM(SUM(SalesAmount)) OVER ()
FROM dbo.FactResellerSales
GROUP BY YEAR(OrderDate);
```

	Year	Sales Amount	GrandTotal
1	2010	489328,5787	80450596,9823
2	2013	33574834,1572	80450596,9823
3	2011	18192802,7143	80450596,9823
4	2012	28193631,5321	80450596,9823

Common patterns



Median

```
SELECT DISTINCT
      DepartmentName
     ,MedianRate = PERCENTILE_CONT(0.5)
                   WITHIN GROUP(ORDER BY BaseRate)
                   OVER (PARTITION BY DepartmentName)
FROM AdventureWorksDW2019.dbo.DimEmployee
ORDER BY MedianRate DESC;
```

	DepartmentName	MedianRate
1	Executive	60,0962
2	Research and Development	41,6731
3	Engineering	34,375
4	Information Services	32,4519
5	Tool Design	28,8462
6	Sales	23,0769
7	Finance	19
8	Purchasing	18,2692
9	Human Resources	17,42785
10	Document Control	16,8269
11	Production Control	16
12	Marketing	14,4231
13	Production	12,45
14	Quality Assurance	10,5769
15	Facilities and Maintenance	9,25
16	Shipping and Receiving	9,25

RANGE vs ROWS

```
SELECT
      [Group]
     ,[Value]
     ,RowsTest    = SUM([Value]) OVER (PARTITION BY [Group] ORDER BY [Value]
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
     ,RangeTest   = SUM([Value]) OVER (PARTITION BY [Group] ORDER BY [Value]
                        RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
FROM Test.dbo.TestRangeRows;
```

	Group	Value	RowsTest	RangeTest
1	A	1	1	4
2	A	1	2	4
3	A	1	3	4
4	A	1	4	4
5	B	1	1	5
6	B	1	2	5
7	B	1	3	5
8	B	1	4	5
9	B	1	5	5

Dealing with duplicates

```
WITH cte_src AS
(
    SELECT
         ID
        ,EmployeeKey
        ,EmployeeName
        ,InsertDate
        ,RID = ROW_NUMBER() OVER (PARTITION BY EmployeeKey ORDER BY InsertDate)
    FROM Test.dbo.EmployeeDuplicate
)
DELETE FROM Test.dbo.EmployeeDuplicate
FROM Test.dbo.EmployeeDuplicate ed
JOIN cte_src                    c ON ed.ID = c.ID
WHERE c.RID <> 1;
```

Dealing with duplicates

```
WITH cte_src AS
(
    SELECT
         ID
        ,EmployeeKey
        ,EmployeeName
        ,InsertDate
        ,RID = ROW_NUMBER() OVER (PARTITION BY EmployeeKey ORDER BY InsertDate)
    FROM Test.dbo.EmployeeDuplicate
)
DELETE FROM Test.dbo.EmployeeDuplicate
FROM Test.dbo.EmployeeDuplicate ed
JOIN cte_src                    c ON ed.ID = c.ID
WHERE c.RID <> 1;
```

Qualify?

```
SELECT
      ID
     ,EmployeeKey
     ,EmployeeName
     ,InsertDate
     ,RID = ROW_NUMBER() OVER (PARTITION BY EmployeeKey ORDER BY InsertDate)
FROM Test.dbo.EmployeeDuplicate
QUALIFY RID = 1;
```

Does not exist in SQL Server but is included MS Fabric Warehouse.
It's also available on other relational databases, such as Databricks & Snowflake

Tally table

```
WITH T0 AS (SELECT N FROM (VALUES (1),(1)) AS tmp(N))
    ,T1 AS (SELECT N = 1 FROM T0 AS a CROSS JOIN T0 AS b)
    ,T2 AS (SELECT N = 1 FROM T1 AS a CROSS JOIN T1 AS b)
    ,T3 AS (SELECT N = 1 FROM T2 AS a CROSS JOIN T2 AS b)
    ,T4 AS (SELECT N = 1 FROM T3 AS a CROSS JOIN T3 AS b)
SELECT DATEADD(DAY,ROW_NUMBER() OVER
          (ORDER BY (SELECT NULL)) -1,'2024-01-01')
FROM T4;
```

this will quickly generate 65K rows

use it to replace loops with set-based logic

	MyDate
1	2024-01-01 00:00:00.000
2	2024-01-02 00:00:00.000
3	2024-01-03 00:00:00.000
4	2024-01-04 00:00:00.000
5	2024-01-05 00:00:00.000
6	2024-01-06 00:00:00.000
7	2024-01-07 00:00:00.000
8	2024-01-08 00:00:00.000
9	2024-01-09 00:00:00.000
10	2024-01-10 00:00:00.000
11	2024-01-11 00:00:00.000
12	2024-01-12 00:00:00.000
13	2024-01-13 00:00:00.000
14	2024-01-14 00:00:00.000
15	2024-01-15 00:00:00.000
16	2024-01-16 00:00:00.000
17	2024-01-17 00:00:00.000

DEMO

Inspect other rows

```
WITH CTE_PY AS
(
  SELECT
     [Year]             = YEAR(OrderDate)
    ,[Sales Amount]     = SUM(SalesAmount)
    ,[Sales Amount PY] = LAG(SUM(SalesAmount)) OVER (ORDER BY YEAR(OrderDate))
  FROM AdventureWorksDW2019.dbo.FactResellerSales
  GROUP BY YEAR(OrderDate)
)
SELECT
    [Year]
  ,[Sales Amount]
  ,[Sales Amount Previous Year]
  ,[YoY Growth] = 100.0 * ([Sales Amount] - [Sales Amount Previous Year])
                        / [Sales Amount Previous Year]
FROM CTE_PY
ORDER BY [Year];
```

	Year	Sales Amount	Sales Amount Previous Year	YoY Growth
1	2010	489328,5787	NULL	NULL
2	2011	18192802,7143	489328,5787	3617,911339377080204
3	2012	28193631,5321	18192802,7143	54,971347597471044
4	2013	33574834,1572	28193631,5321	19,086589178741322

Inspect other rows

```
WITH CTE_PY AS
(
  SELECT
     [Year]             = YEAR(OrderDate)
    ,[Sales Amount]     = SUM(SalesAmount)
    ,[Sales Amount PY] = LAG(SUM(SalesAmount)) OVER (ORDER BY YEAR(OrderDate))
  FROM AdventureWorksDW2019.dbo.FactResellerSales
  GROUP BY YEAR(OrderDate)
)
SELECT
   [Year]
  ,[Sales Amount]
  ,[Sales Amount Previous Year]
  ,[YoY Growth] = 100.0 * ([Sales Amount] - [Sales Amount Previous Year])
                        / [Sales Amount Previous Year]
FROM CTE_PY
ORDER BY [Year];
```

	Year	Sales Amount	Sales Amount Previous Year	YoY Growth
1	2010	489328,5787	NULL	NULL
2	2011	18192802,7143	489328,5787	3617,911339377080204
3	2012	28193631,5321	18192802,7143	54,971347597471044
4	2013	33574834,1572	28193631,5321	19,086589178741322

Previous Non-NULL value

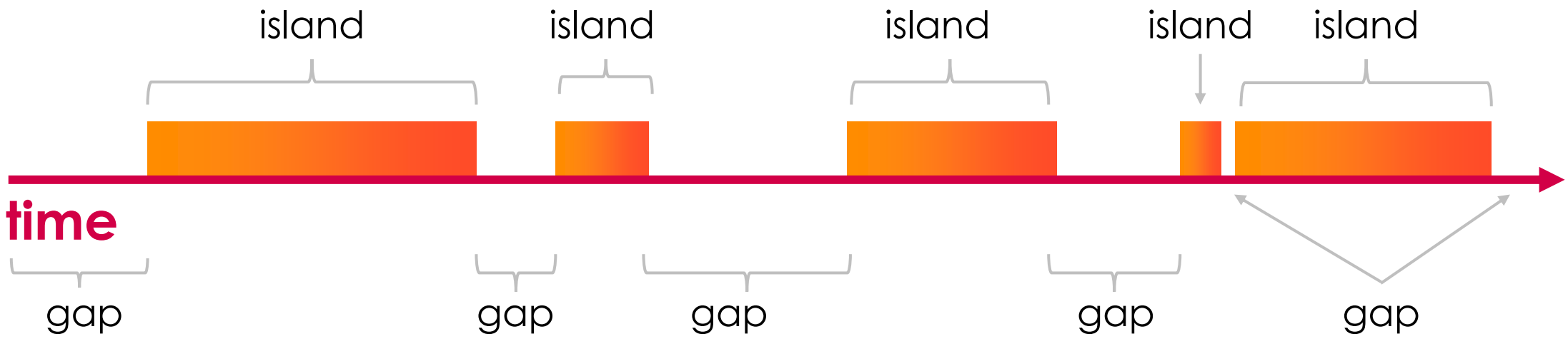
```
SELECT
      ID
     ,ColA
     ,previousnonnull =
         LAG(ColA) IGNORE NULLS
            OVER (ORDER BY ID)
FROM CTE_sampledata;
```

Results | Messages

	ID	ColA	previousnonnull
1	1	5	NULL
2	2	NULL	5
3	3	-10	5
4	4	20	-10
5	5	NULL	20
6	6	NULL	20
7	7	NULL	20
8	8	NULL	20
9	9	25	20
10	10	30	25

Gaps & islands problem

Common analytics problem



Gaps & islands problem

Gaps = holes in the sequence

- how many times did a server go offline, and for how long?
- when did a student not attend class?
- when was a consultant not on a project?

Islands = ranges of consecutive values

- how long was someone visiting a website, and did they return?
- was an employee sick for longer than a day? Did the sick period surpass a certain limit, so they're now on extended sick leave?
- how long was a consultant allocated to a project?

Gaps example

ID	ServerName	DayOnline
1	MyServer	01/05/2024
2	MyServer	02/05/2024
3	MyServer	03/05/2024
4	MyServer	04/05/2024
5	MyServer	05/05/2024
6	MyServer	07/05/2024
7	MyServer	08/05/2024
8	MyServer	09/05/2024
9	MyServer	15/05/2024
10	MyServer	16/05/2024
11	MyServer	17/05/2024
12	MyServer	19/05/2024
13	MyServer	20/05/2024



ServerName	DayOffline
MyServer	06/05/2024
MyServer	10/05/2024
MyServer	11/05/2024
MyServer	12/05/2024
MyServer	13/05/2024
MyServer	14/05/2024
MyServer	18/05/2024



ServerName	DaysOffline
MyServer	1
MyServer	5
MyServer	1

Find the next start date

```
SELECT
      ID
     ,[current] = DayOnline
     ,[next]    = LEAD(DayOnline) OVER (ORDER BY DayOnline)
FROM dbo.[Server];
```

	ID	current	next
1	1	2024-05-01	2024-05-02
2	2	2024-05-02	2024-05-03
3	3	2024-05-03	2024-05-04
4	4	2024-05-04	2024-05-05
5	5	2024-05-05	2024-05-07
6	6	2024-05-07	2024-05-08
7	7	2024-05-08	2024-05-09
8	8	2024-05-09	2024-05-15
9	9	2024-05-15	2024-05-16
10	10	2024-05-16	2024-05-17
11	11	2024-05-17	2024-05-19
12	12	2024-05-19	2024-05-20
13	13	2024-05-20	NULL

Find gaps > 1

same as previous slide

```
WITH cte_lead AS
(
    SELECT
         [current] = DayOnline
        ,[next]    = LEAD(DayOnline) OVER (ORDER BY DayOnline)
    FROM dbo.[Server]
)
SELECT
     gapStart = DATEADD(DAY, 1, [current])
    ,gapEnd   = DATEADD(DAY,-1, [next])
    ,gapWidth = DATEDIFF(DAY, [current], DATEADD(DAY,-1, [next]))
FROM cte_lead
WHERE DATEDIFF(DAY, [current], [next]) > 1;
```

identify the gaps

	ID	current	next
1	1	2024-05-01	2024-05-02
2	2	2024-05-02	2024-05-03
3	3	2024-05-03	2024-05-04
4	4	2024-05-04	2024-05-05
5	5	2024-05-05	2024-05-07
6	6	2024-05-07	2024-05-08
7	7	2024-05-08	2024-05-09
8	8	2024-05-09	2024-05-15
9	9	2024-05-15	2024-05-16
10	10	2024-05-16	2024-05-17
11	11	2024-05-17	2024-05-19
12	12	2024-05-19	2024-05-20
13	13	2024-05-20	NULL

	gapStart	gapEnd	gapWidth
1	2024-05-06	2024-05-06	1
2	2024-05-10	2024-05-14	5
3	2024-05-18	2024-05-18	1

Islands example

ID	ServerName	DayOnline
1	MyServer	01/05/2024
2	MyServer	02/05/2024
3	MyServer	03/05/2024
4	MyServer	04/05/2024
5	MyServer	05/05/2024
6	MyServer	07/05/2024
7	MyServer	08/05/2024
8	MyServer	09/05/2024
9	MyServer	15/05/2024
10	MyServer	16/05/2024
11	MyServer	17/05/2024
12	MyServer	19/05/2024
13	MyServer	20/05/2024



ServerName	StartDate	EndDate
MyServer	01/05/2024	05/05/2024
MyServer	07/05/2024	09/05/2024
MyServer	15/05/2024	17/05/2024
MyServer	19/05/2024	20/05/2024

Calculate dense rank

```
SELECT
      DayOnline
     ,DRank = DENSE_RANK() OVER (PARTITION BY ServerName
                                 ORDER BY DayOnline)
FROM dbo.[Server];
```

	DayOnline	DRank
1	2024-05-01	1
2	2024-05-02	2
3	2024-05-03	3
4	2024-05-04	4
5	2024-05-05	5
6	2024-05-07	6
7	2024-05-08	7
8	2024-05-09	8
9	2024-05-15	9
10	2024-05-16	10
11	2024-05-17	11
12	2024-05-19	12
13	2024-05-20	13

Dense rank =
rank with no gaps, ties have the same rank

Subtract rank from date

```
SELECT
  DayOnline
 ,[Group] = DATEADD(DAY,-1 * DENSE_RANK() OVER
                                  (PARTITION BY ServerName
                               ORDER BY DayOnline)
                     , DayOnline)
FROM dbo.[Server];
```

	DayOnline	Group
1	2024-05-01	2024-04-30
2	2024-05-02	2024-04-30
3	2024-05-03	2024-04-30
4	2024-05-04	2024-04-30
5	2024-05-05	2024-04-30
6	2024-05-07	2024-05-01
7	2024-05-08	2024-05-01
8	2024-05-09	2024-05-01
9	2024-05-15	2024-05-06
10	2024-05-16	2024-05-06
11	2024-05-17	2024-05-06
12	2024-05-19	2024-05-07
13	2024-05-20	2024-05-07

Aggregate

```
WITH cte_groups AS
(
    SELECT
      DayOnline
     ,[Group] = DATEADD(DAY, -1 * DENSE_RANK() OVER
                                 (PARTITION BY ServerName
                                  ORDER BY DayOnline)
                        , DayOnline)
    FROM dbo.[Server]
)
SELECT
      StartDate = MIN(DayOnline)
     ,EndDate   = MAX(DayOnline)
FROM cte_groups
GROUP BY [Group];
```

same as previous slide

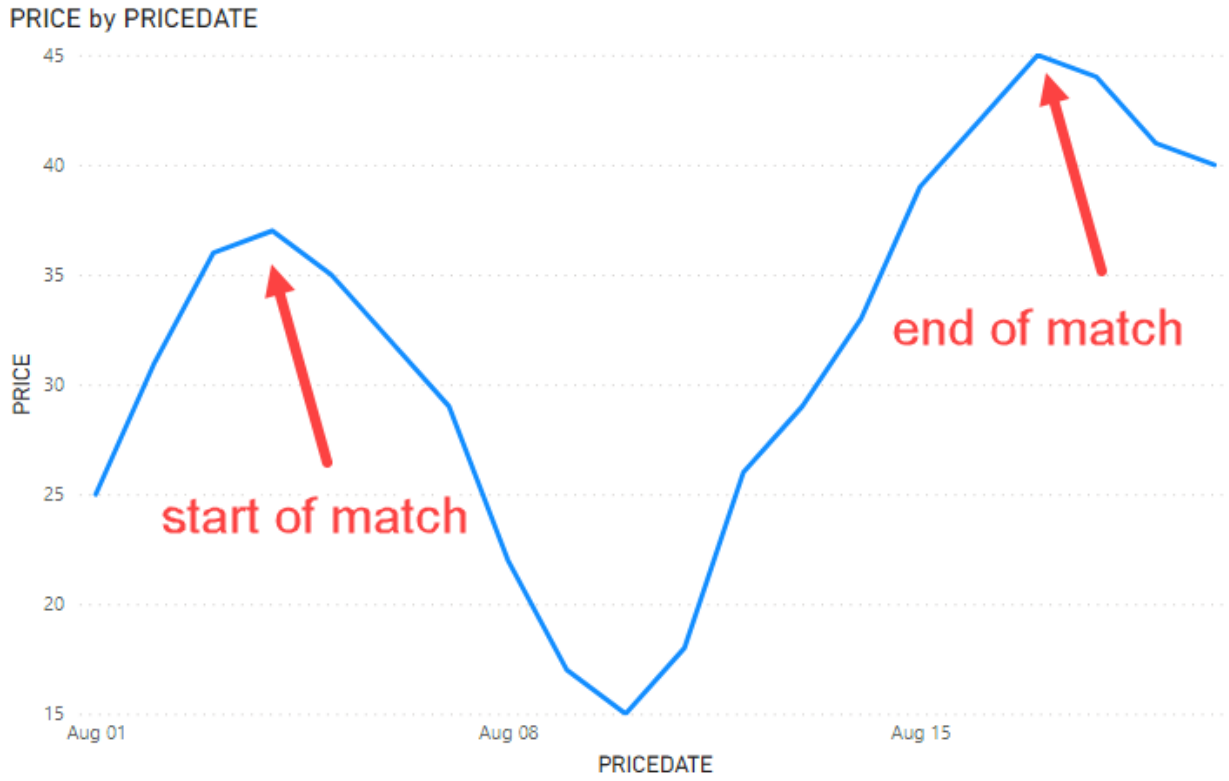
aggregate
to get range

	StartDate	EndDate
1	2024-05-01	2024-05-05
2	2024-05-07	2024-05-09
3	2024-05-15	2024-05-17
4	2024-05-19	2024-05-20

DEMO

Row Pattern Recognition (RPR)

- Advanced window function used to apply “regex over rows” to find patterns in your data
- For example, V or W shapes in the stock market



Row Pattern Recognition (RPR)

- **Does not exist in SQL Server or MS Fabric databases**
- It is available in Stream Analytics and MS Fabric Eventstream
- And on other relational systems like Databricks and Snowflake



Row Pattern Recognition (RPR)

```
SELECT *
FROM dbo.StockPrice
MATCH_RECOGNIZE
(
    PARTITION BY CompanyName
    ORDER BY PriceDate
    MEASURES
         MATCH_NUMBER()    AS match_number
        ,FIRST(PriceDate)  AS start_date
        ,LAST(PriceDate)   AS end_date
        ,COUNT(*)          AS rows_in_sequence
        ,COUNT(B.*)        AS num_decreases
        ,COUNT(C.*)        AS num_increases
        ,MAX(Price)        AS max_price
        ,MIN(Price)        AS min_price
    ONE ROW PER MATCH
    AFTER MATCH SKIP TO LAST C
    PATTERN(A B+ C+)
    DEFINE
         B AS Price < LAG(Price)
        ,C AS Price > LAG(Price)
)
ORDER BY CompanyName, match_number;
```



DEMO

Performance

Index suggestions

POC index = Partitioning, Ordering, Covering

```
window function
OVER (
    [PARTITION BY expression]
    [ORDER BY expression]
    [ROWS | RANGE BETWEEN frame bound AND frame bound]
     )
```

first columns of the index

other columns of the index

+ other columns needed for SELECT clause as INCLUDED

Other tricks?

- Columnstore for large tables
  - take advantage of batch mode
- `APPLY` trick to force parallelism
  - for the adventurous people amongst us
- don't forget the defaults
  - `ROWS` vs `RANGE`
  - frame extent
- `ROW_NUMBER() OVER (ORDER BY (SELECT NULL))`
  - when the sorting doesn't matter

DEMO

Conclusion

window functions are awesome

They can solve a wide range of common business analyses problems

gaps & island is a common pattern that window functions solve elegantly

Code is more elegant and efficient

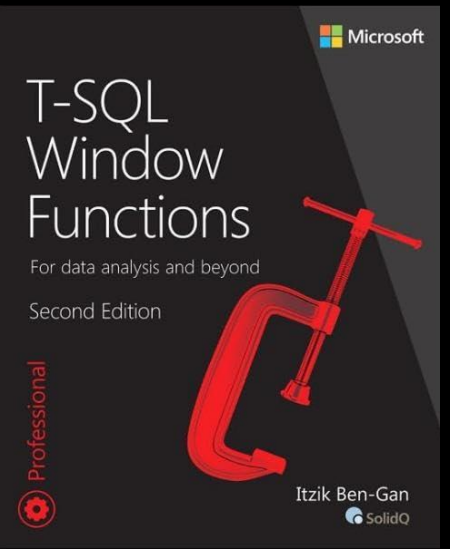
They have good performance out-of-the-box

be aware of the defaults



Resources

- T-SQL OVER clause
- Window Functions Tutorial
- Logical Query Processing
- Number series generator challenge solutions
- Row Pattern Recognition in Snowflake
- Previous NON NULL value



THANK YOU

www.ae.be
inspire@ae.be

+32 16 39 30 60
Interleuvenlaan 27b
3001 Heverlee



Spice up your SQL - Intro to Window Functions

Session Feedback 💖



Spice up your SQL - Intro to Window Functions